

LAB 4 REPORT

SysTick Timer

Name: Michael McCorkell

Student ID: N01500049

Course: Microcontrollers – Winter 2026

Instructor: Nilufar Poshtiban

Date: March 27, 2026

Abstract

In this lab, the TM4C123GH6PM's SysTick timer was used to do three things: flash an LED at 2 Hz with a 500 ms delay, stretch that delay to 2 seconds (which quickly approached a hardware limit), and measure the length of time between two button presses. There was also a challenge question on why busy-wait time fails while attempting to run many tasks at once. Overall, polling SysTick worked well for simple tasks, but the challenge question demonstrated that when timing accuracy is crucial, you should use interrupts.

Introduction

The SysTick timer is a 24-bit counter that is built directly into the ARM Cortex-M4 core, so it is always available without the need to activate a separate peripheral. It counts down from the value contained in the RELOAD register and automatically reloads when it gets to zero. You may either poll the COUNT flag in the CTRL register to check when it has expired, or you can program the interrupt to occur automatically. At 16 MHz, each tick lasts 62.5 ns, therefore the longest single countdown is around 1.049 seconds before it wraps - something that is critical in Activity 2.

Before starting, the pre-lab directed us to change `#define CLOCK_SETUP 1` to `#define CLOCK_SETUP 0` in `system_TM4C123.c`, allowing the board to run on the default 16 MHz internal oscillator. This was essential since all LOAD computations depend on knowing the exact clock frequency.

Objectives

- Configure the SysTick timer to produce a precise 500 ms delay and blink the blue LED at 2 Hz.
- Extend the delay to 2 seconds and explain the limitation of the 24-bit SysTick counter at 16 MHz.
- Measure the elapsed time between two button presses using SysTick with overflow tracking.
- Analyse the limitations of busy-wait SysTick delays for periodic tasks and propose an interrupt-driven alternative.
- Answer the laboratory questions relating to register configuration, mathematics, and timing accuracy.

Equipment

- TI TM4C123GH6PM Tiva C LaunchPad
- USB cable / USB-to-Serial interface
- Computer with Keil MDK IDE
- PuTTY terminal software
- Connecting wires

Activity 1 – 500 ms Delay and 2 Hz LED Blink

Configuration

To begin, the Port F clock was activated using SYSCTL->RCGCGPIO, and then the peripheral-ready bit was checked before accessing any of the GPIO registers; missing that step created problems in a prior experiment, so it is now a habit. PF2 was configured as a digital output for the blue LED.

CTRL was set to 0 to disable SysTick, then 0x05 to re-enable it using the system clock and no interrupt. The real delay is caused by a utility function called waitSystick(), which repeatedly runs a 'delay' loop. Each pass loads 15999 into the LOAD register, which is computed as $16,000,000 \text{ Hz} / 1000 \text{ ms} = 16,000 \text{ ticks per millisecond}$, less one since the timer accepts zero as a valid value. Then VAL is cleared to force an immediate reload, and the code just rotates bit 16 of CTRL until the COUNT flag is high.

To turn on and off the LED, the main loop XORs PF2 and calls waitSystick(250), resulting in a full cycle of 500 ms, or 2 Hz.

Code Excerpt – Activity 1

```
SysTick->CTRL = 0;           // disable during setup
SysTick->CTRL = 0x05;        // enable, system clock, no interrupt

while(1){
    GPIOF->DATA ^= 0x04;     // toggle blue LED (PF2)
    waitSystick(250);        // 250 ms delay
}

void waitSystick(unsigned long delay){
    for(unsigned long i = 0; i < delay; i++){
        SysTick->LOAD = 15999; // 1 ms @ 16 MHz
        SysTick->VAL = 0;      // clear counter
        while((SysTick->CTRL & 0x10000) == 0){} // wait for COUNT
    }
}
```

Result

On the board, the blue LED blinks at a frequency of 2 Hz. PuTTY also returned "Activity 1: Ongoing" via UART when the function started, indicating that the application was running and had not stopped during initialization.

Activity 2 – 2-Second Blink Period

SysTick Limitation at 16 MHz

The first thing that comes to mind for a 2-second delay is to set LOAD to a large value, but SysTick is only 24 bits wide. The maximum value it can hold is 16,777,215; however, at 16 MHz, it only has around 1.049 seconds before looping around. So, a single countdown is insufficient.

$$16,777,216 \text{ ticks} \div 16,000,000 \text{ Hz} \approx \mathbf{1.049 \text{ seconds}}$$

To go around this, the LOAD was set to 15,999,999 (equal to one second at 16 MHz: 16,000,000 - 1), and the loop was invoked with waitSystick(2), causing it to execute twice. Two one-second repetitions yield the required two-second delay.

Code Excerpt – Activity 2

```
void waitSystick(unsigned long delay){
    for(unsigned long i = 0; i < delay; i++){
        SysTick->LOAD = 15999999; // 1 s @ 16 MHz
        SysTick->VAL = 0;
        while((SysTick->CTRL & 0x10000) == 0){}
    }
}

// In Activity2():
while(1){
    GPIOF->DATA ^= 0x04;
    waitSystick(2); // 2 x 1-second iterations = 2 s
}
```

Limitation Summary

At 16 MHz, SysTick can only operate for approximately 1.049 seconds. The solution in this case was to loop many 1-second countdowns, but another option would be to utilize one of the general-purpose timers, which contain greater counts and prescalers for comparable situations.

Activity 3 – Time Measurement Between Button Presses

Design Overview

For this activity, SysTick was set to free-running mode with `LOAD = 0x00FFFFFF`, the highest value that the 24-bit register can hold. The interrupt was also enabled, such that when the counter wraps, a global variable called `systick_overflows` is incremented within the `SysTick_Handler` ISR. This enables for the reconstruction of elapsed ticks even if the user pushes a button after a delay of around a second.

When you press SW1 (PF4), the current VAL reading and overflow count are retained. The same thing happens when you press SW2 (PF0). SysTick counts down, therefore the tick difference must account for the direction:

```
tickDiff = (secondOverflow - firstOverflow) * (0x00FFFFFF + 1) + (firstTime - secondTime);
```

Then dividing by 16 converts ticks to microseconds since there are 16 ticks per microsecond at 16 MHz:

```
time_us = tickDiff / 16;
```

GPIO Configuration

Port F was configured with PF4 and PF0 as digital inputs, pull-ups enabled because the buttons are active-low. PF0 is a locked pin on the TM4C123, therefore the unlock sequence - writing `0x4C4F434B` to `GPIOF->LOCK` and then setting bit 0 of `CR` - is required before the pin can be controlled.

Code Excerpt – Activity 3

```
static void SysTick_FreeRunInit(void) {
    SysTick->CTRL = 0;
    SysTick->LOAD = 0x00FFFFFF;    // max 24-bit reload
    SysTick->VAL = 0;
    SysTick->CTRL = 0x07;          // enable, sysclk, interrupt on
}

void SysTick_Handler(void) {
    systick_overflows++;
}

// Capture on SW1 press:
firstOverflow = systick_overflows;
firstTime     = SysTick->VAL;

// Capture on SW2 press:
secondOverflow = systick_overflows;
secondTime     = SysTick->VAL;

tickDiff = (secondOverflow - firstOverflow) * (0x00FFFFFF + 1)
           + (firstTime - secondTime);
```

```
time_us = tickDiff / 16;
```

Result

After pressing SW1 and SW2, PuTTY receives the elapsed time in microseconds in the format "Time(us): XXXXXX". The wait-for-release calls after each press prevented the bounce from being counted as a second press, which was previously not allowed.

Activity 4 – Challenge: Periodic Task Scheduling Analysis

The Student's Code

```
while(1){
    delay_ms(1);          // using SysTick busy-wait
    TaskA();
    if(counter == 100){
        TaskB();
        counter = 0;
    }
    counter++;
}
```

Question 1 – Will Task A execute exactly every 1 ms?

No, it will not. The `delay_ms(1)` method waits 1 ms before running `TaskA()`. Whatever time `TaskA()` takes is added to the period, therefore Task A is always little late. The delay does not compensate for execution time, but rather adds to it.

Question 2 – Actual period when TaskA() takes 0.2 ms

1.0 ms (delay) + 0.2 ms (TaskA execution) = 1.2 ms actual period.

Question 3 – Will Task B run exactly 10 times after 1 second?

No. Since each loop iteration is actually 1.2 ms instead of 1.0 ms, Task B fires every 100×1.2 ms = 120 ms instead of every 100 ms. After 1 second that's only about 8 executions, not 10. The error comes from the fact that TaskA's runtime is not accounted for when counting iterations.

Question 4 – Main limitation of busy-wait SysTick delays

The CPU is entirely halted when spinning on the COUNT flag; nothing else can execute during that period. And, because task execution time is added to the delay, each task is slightly late, which accumulates over time. It wanders more and more as the system continues to operate.

Question 5 – Better Design Using SysTick Interrupts

Instead of performing the jobs straight within the ISR, setup SysTick to emit an interrupt every 1 ms and utilize it to set flags. The main loop just checks the flags and does the tasks when they are set. This manner, the timing is determined by the hardware rather than how long the jobs take:

```
volatile uint32_t tickCount = 0;
volatile uint8_t flagA = 0, flagB = 0;

void SysTick_Handler(void){
    tickCount++;
    flagA = 1;                      // set every 1 ms
    if(tickCount % 100 == 0)
```

```
        flagB = 1;                // set every 100 ms
    }

    int main(void) {
        SysTick_Config(16000);      // 1 ms period @ 16 MHz
        while(1) {
            if(flagA) { flagA = 0; TaskA(); }
            if(flagB) { flagB = 0; TaskB(); }
        }
    }
}
```

The essential distinction is that tickCount is incremented depending on a hardware interrupt rather than the length of the loop body. So, even if TaskA() is a little delayed on one iteration, the following interrupt fires precisely 1 ms later and sets the flag again. There's no drift.

Lab Questions

Question 1 – SysTick configuration for 500 ms delay at 16 MHz

The setup was as follows: first, write 0 to CTRL to deactivate SysTick, then write 0x05 to CTRL to re-enable it with the system clock and no interrupt (bit 2 = clock source, bit 0 = enable, bit 1 left off for no interrupt). Inside the delay loop, LOAD is reset each iteration, and VAL is cleared to start a new countdown.

The math for the LOAD value:

At 16 MHz, one tick = $1 / 16,000,000 \text{ s} = 62.5 \text{ ns}$

Ticks needed for 1 ms = $1,000,000 \text{ ns} / 62.5 \text{ ns} = 16,000 \text{ ticks}$

LOAD = $16,000 - 1 = 15,999$ (because the counter counts 0 through LOAD, inclusive)

500 ms = 500 iterations of the loop, but for blinking the LED each half-period is 250 ms, so waitSysTick(250) is called each toggle.

Question 2 – Modifying the code for 10 Hz blinking

10 Hz means the LED toggles 10 times per second, so each half-cycle (on or off) is 50 ms. Just change waitSysTick(250) to waitSysTick(50):

```
while(1){
    GPIOF->DATA ^= 0x04;
    waitSysTick(50); // 50 ms half-period → 10 Hz
}
```

Question 3 – Impact of a 10 MHz clock source

With 10 MHz, each tick lasts 100 ns rather than 62.5 ns. So, if you left LOAD at 15,999 (measured for 16 MHz), each loop iteration would be 1.6 ms rather than 1.0 ms. The LED would blink at around 1.25 Hz rather than 2 Hz, which was slower than planned.

To correct it for 10 Hz at 10 MHz, 1 millisecond requires 10,000 ticks, therefore LOAD = 9,999. Then execute waitSysTick(50) for a 50 ms half-period.

Question 4 – Calculating elapsed time from SysTick values

SysTick ticks down, therefore a lower VAL value indicates more time has elapsed. When the counter reaches 0, it reloads from LOAD, and the overflow ISR increases the counter. The total tick count between two captures is:

```
tickDiff = (ov2 - ov1) * (0x00FFFFFF + 1) + (val1 - val2)
```

The reading at the first press is denoted by val1, and at the second by val2. The (val1 - val2) component handles the downward direction. Then divide by 16 yields microseconds since at 16 MHz, there are precisely 16 ticks each microsecond.

```
time_us = tickDiff / 16
```

Question 5 – Handling SysTick rollover for long intervals

Without overflow monitoring, the timer will automatically reload after more than ~1.049 seconds between pushes. At that point, `val2` may be larger than `val1`, and the subtraction produces an inaccurate (or negative) result.

Activity 3 makes use of the ISR's overflow counter as a remedy. Every time SysTick wraps, the count grows by one, with each wrap representing 0x01000000 additional ticks. Even if the user waits for several seconds, the formula will still provide the right answer if the overflow counter fits in 32 bits (which can survive up to 268 seconds). Anything longer than that necessitates the use of a 64-bit counter or a dedicated general-purpose timer.

Results

Activity	Outcome	Key Observation
Activity 1 – 2 Hz LED Blink	Success	Blue LED toggled at 500 ms half-period using <code>LOAD = 15,999</code> iterated 250 times per half-cycle.
Activity 2 – 2-Second Blink	Success	<code>LOAD</code> set to 15,999,999 (1 s per iteration); called twice per toggle for a 2 s period.
Activity 3 – Button Timer	Success	Elapsed time transmitted via UART. Overflow counter correctly extended the measurement range beyond ~1 s.
Activity 4 – Challenge	Analysed	Interrupt-driven flag approach proposed to eliminate timing drift.

Discussion

Activity 1 went rather easily. Polling SysTick for a 1 ms construction block performed nicely, and stacking 250 of them each half-cycle resulted in a clear 2 Hz blink. The UART output in PuTTY made it simple to verify that the function was indeed executing before entering the blink loop.

Activity 2 was when the 24-bit constraint became a practical issue rather than a theoretical one. Setting `LOAD` to maximum and witnessing the math fall short of 2 seconds demonstrated why you can't just enter a large number into the register and expect it to work. Splitting it into two 1-second repeats was a straightforward fix, but it means the CPU is sitting in the busy-wait state for two whole seconds doing nothing else.

Activity 3 was arguably the most involved. Getting the overflow counter right required some consideration because SysTick counts down and you must be cautious about the direction when computing the difference. It's worth noting that the timestamps are taken in the polling loop after the button is pressed, rather than at the precise instant the pin changes. In a more precise approach, you would catch the edge in a GPIO interrupt and timestamp it. However, in this lab, the inaccuracy was insignificant.

The challenge question ties everything together. The student's code in Activity 4 appears to be okay at first glance, but the issue is that `delay_ms(1)` has no idea or cares how long `TaskA()` takes. Each cycle, the period is slightly extended, and this mistake accumulates. Switching to an interrupt-driven technique solves this issue since the hardware timer fires regardless of what the CPU does.

Conclusion

This lab looked at the most typical ways to use SysTick on the TM4C123, such as basic LED flashing, bypassing the 24-bit counter constraint for longer delays, and tracking time between occurrences. The button timer in Activity 3 was the most helpful, and the overflow tracking enabled it to be utilized for periods longer than a second. The challenge question served as a good reminder that polling-based delays look simple but fail rapidly when dealing with activities that are more complex than one. Interrupts are the most effective technique to achieving genuine periodic scheduling.

References

Mazidi, M. A., Chen, S., Naimi, S., & Naimi, S. (2017). TI ARM Peripherals Programming and Interfacing: Programming ARM Cortex-M4 TM4C123G with C. MicroDigitalEd.com.

Texas Instruments. (2014). Tiva TM4C123GH6PM Microcontroller Data Sheet (Rev. E). <https://www.ti.com/lit/ds/spms376e/spms376e.pdf>

ARM Limited. (2010). Cortex-M4 Devices Generic User Guide: SysTick Timer. <https://developer.arm.com/documentation/dui0553/latest>

Writing Advice Home. University of Toronto. <https://advice.writing.utoronto.ca/types-of-writing/lab-report/>